Lab 35 — Secure Remote Access: SSH Hardening and Fail2ban

> **SY0-701 Objective 4.5** — Given a scenario, modify enterprise capabilities to enhance security.
>
> **Domain 4** — Security Operations (28%) · **Time:** ~20 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Replace password logins with key-only SSH, restrict the protocol to modern cryptography, then let Fail2ban turn failed-login logs into automatic firewall blocks against a simulated attacker address.

You'll build: A hardened `sshd` drop-in configuration with a login banner, an Ed25519 key login, and a Fail2ban jail that bans the attacker address `10.10.10.10` in nftables or iptables.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 20 (firewalls) is helpful background; tools installed in step 1.

> **Legal note:** The brute-force simulation targets only the playground itself. Never run login attacks against systems you are not authorised to test (Singapore Computer Misuse Act 1993).

Background

- **Implementation of secure protocols** covers three choices: **protocol selection** (SSH instead of Telnet, SFTP instead of FTP), **port selection** (well-known port 22, or a non-standard port that only reduces noise) and **transport method** (an encrypted channel with strong key exchange, cipher and MAC).
- Public-key authentication never sends a reusable secret. The private key stays on the client and the server stores only the public key, which defeats password spraying and credential stuffing.
- **Fail2ban** reads authentication logs, counts failures per source within `findtime`, and after `maxretry` failures inserts a **firewall rule** that drops that source for `bantime`. Log-driven blocking like this is a simple form of an intrusion prevention response.
- A **login banner** is a directive and deterrent control, and it supports legal action by stating that access is monitored.
- Always validate the configuration (`sshd -t`) and keep a working session open before restarting SSH, so that you do not lock yourself out.

Steps

1. Install the tools

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y openssh-server
openssh-client fail2ban python3-systemd nftables
```

```
systemctl enable --now ssh
systemctl is-active ssh
```

2. Audit the current exposure

Record the effective SSH settings before you change anything, and confirm that no cleartext remote-access protocol is listening (Telnet on 23, rlogin on 513).

```
sshd -T | grep -Ei
'^(port|permitrootlogin|passwordauthentication|kbdinteractiveauthentication|pubkeyauthenti
cation|maxauthtries|x11forwarding) '
ss -tlnp | grep -E ':(22|23|513) ' || true
ss -tln | grep -qE ':(23|513) ' && echo "WARNING: cleartext remote access listening" ||
echo "OK: no Telnet or rlogin listener"
```

3. Create and authorise an Ed25519 key

```
mkdir -p -m 700 ~/.ssh
ssh-keygen -t ed25519 -f ~/.ssh/secplus_ed25519 -N "" -C "secplus-lab-key" -q
cat ~/.ssh/secplus_ed25519.pub >> ~/.ssh/authorized_keys && chmod 600
~/.ssh/authorized_keys
ssh-keygen -lf ~/.ssh/secplus_ed25519.pub
```

4. Harden sshd with a drop-in file

Files in `sshd_config.d` are read in name order and the **first** value set wins, so the `01-` prefix makes these settings take priority. `PermitRootLogin prohibit-password` keeps key-based root access for this playground; in production you would set `no` and log in as a named user.

```
cat > /etc/issue.net <<'EOF'
AUTHORISED ACCESS ONLY. Activity on this system is monitored and recorded.
Unauthorised access is an offence under the Computer Misuse Act 1993.
EOF
cat > /etc/ssh/sshd_config.d/01-secplus-hardening.conf <<'EOF'
PermitRootLogin prohibit-password
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
PermitEmptyPasswords no
MaxAuthTries 3
LoginGraceTime 20
X11Forwarding no
ClientAliveInterval 300
ClientAliveCountMax 2
KexAlgorithms curve25519-sha256,curve25519-sha256@libssh.org
Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com,aes128-gcm@openssh.com
MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com
Banner /etc/issue.net
EOF
sshd -t && systemctl restart ssh && echo "sshd configuration valid and reloaded"
sshd -T | grep -Ei '^(passwordauthentication|maxauthtries|ciphers|kexalgorithms) '
```

5. Verify key-only login and the negotiated crypto

The key login must succeed, the password method must be refused, and the verbose output shows the modern key exchange and cipher actually in use.

```
ssh -i ~/.ssh/secplus_ed25519 -o StrictHostKeyChecking=accept-new -o BatchMode=yes
root@127.0.0.1 'echo KEY-LOGIN-OK'
ssh -o PubkeyAuthentication=no -o PreferredAuthentications=password -o BatchMode=yes
root@127.0.0.1 true 2>&1 | tail -1
ssh -v -i ~/.ssh/secplus_ed25519 -o BatchMode=yes root@127.0.0.1 true 2>&1 | grep -m2 -E
'kex: algorithm:|kex: server->client cipher'
```

```
KEY-LOGIN-OK
root@127.0.0.1: Permission denied (publickey).
debug1: kex: algorithm: curve25519-sha256
debug1: kex: server->client cipher: chacha20-poly1305@openssh.com MAC: <implicit>
compression: none
```

6. Configure the Fail2ban jail

Add a lab "attacker" address on the loopback interface so that you can ban it without cutting off your own session. Fail2ban normally ignores the host's own addresses, so this jail switches that off for the lab.

```
ip addr add 10.10.10.10/32 dev lo 2>/dev/null || true
cat > /etc/fail2ban/jail.d/sshd-secplus.local <<'EOF'
[sshd]
enabled    = true
backend    = systemd
port       = ssh
maxretry   = 3
findtime   = 10m
bantime    = 1h
ignoreself = false
ignoreip   =
EOF
systemctl restart fail2ban && sleep 3
fail2ban-client status sshd
```

7. Simulate a brute-force attempt from the attacker address

Bind the SSH client to `10.10.10.10` and try several non-existent users. Each attempt is logged as an invalid user, and on the third failure Fail2ban bans the source.

```
for u in admin oracle test guest ubuntu; do
  ssh -b 10.10.10.10 -o BatchMode=yes -o StrictHostKeyChecking=no -o ConnectTimeout=3
"$u@10.10.10.10" true 2>/dev/null
done
sleep 5
fail2ban-client status sshd | grep -E 'Currently failed|Total failed|Banned IP list'
```

```
   |- Currently failed: 0
   |- Total failed: 5
   `- Banned IP list:   10.10.10.10
```

8. Confirm the firewall block, then unban

The ban is a real packet-filter rule. From the banned address the connection now times out, while key login from `127.0.0.1` still works. Finish with the administrator override.

```
nft list ruleset 2>/dev/null | grep -B3 -A3 '10.10.10.10' || iptables -S | grep -E
'f2b|10.10.10.10'
ssh -b 10.10.10.10 -o BatchMode=yes -o ConnectTimeout=3 root@10.10.10.10 true 2>&1 | tail
-1
ssh -i ~/.ssh/secplus_ed25519 -o BatchMode=yes root@127.0.0.1 'echo STILL-OK-FROM-
LOCALHOST'
fail2ban-client set sshd unbanip 10.10.10.10
fail2ban-client status sshd | grep 'Banned IP list'
```

Test it

```
sshd -T | grep -E '^passwordauthentication no$' && echo "PASS key-only"
ssh -i ~/.ssh/secplus_ed25519 -o BatchMode=yes root@127.0.0.1 'echo PASS key-login'
for u in a b c d; do ssh -b 10.10.10.10 -o BatchMode=yes -o ConnectTimeout=3
"$u@10.10.10.10" true 2>/dev/null; done; sleep 5
fail2ban-client status sshd | grep -q '10.10.10.10' && echo "PASS ban active"
fail2ban-client set sshd unbanip 10.10.10.10
```

Expected: `PASS key-only`, `PASS key-login` and `PASS ban active`.

Troubleshooting

Symptom	Fix
`sshd -t` reports `Bad SSH2 cipher spec`	An algorithm name is misspelt or unsupported. Run `ssh -Q cipher`, `ssh -Q kex` and `ssh -Q mac` and use only names listed there.
Fail2ban never bans 10.10.10.10	Check `fail2ban-client get sshd ignoreself` (must be `false`) and `journalctl -u ssh -n 10` for `Invalid user` lines. If the journal is empty, set `backend = auto` and `logpath = /var/log/auth.log`.
`Missing systemd journal backend`	Install `python3-systemd` (step 1) and restart fail2ban.

Challenge

Restrict SSH to a single management subnet with an nftables rule (allow `tcp dport 22` only from `10.10.50.0/24`), and write down which of the three secure-protocol choices (protocol, port, transport) each of your changes in this lab addressed.

Reflection

Moving SSH to port 2222 cut the number of failed logins by 95 percent. Has the risk changed, and how does that compare with the controls you applied in steps 4 and 6?

Exam tips

- Secure replacements: Telnet to SSH (22), FTP to SFTP (22) or FTPS (990), HTTP to HTTPS (443), LDAP to LDAPS (636), SNMPv1/v2c to SNMPv3.
- Changing the port is obscurity, not security. It cuts noise but stops no targeted attacker.
- Fail2ban-style blocking is a log-driven, automated response, which makes it detective plus preventive.
- Key-based authentication defeats password spraying and brute force; MaxAuthTries and lockout slow online guessing.

References

- Mozilla OpenSSH security guidelines — https://infosec.mozilla.org/guidelines/openssh
- OpenBSD sshd_config manual — https://man.openbsd.org/sshd_config
- Fail2ban documentation — https://github.com/fail2ban/fail2ban/wiki
- CIS Ubuntu Linux Benchmark, SSH server section — https://www.cisecurity.org/benchmark/ubuntu_linux
- MITRE ATT&CK T1110 Brute Force — https://attack.mitre.org/techniques/T1110/
